

APM1110_FA5_Pinili_Nicklaus

Nicklaus Vincent M. Pinili

2026-09-18

Github link <https://github.com/Notcheesechess/APM1110-FA5>

1. (Discrete Random Variables) [7 Points] - A discrete random variable X has the following probability mass function (PMF)

```{r exer1}

**$P$  means probability and  $X$  (detonated as egg as  $R$  variable) is the discrete random variable. The code below is basically the possible values of  $X$ .**

```
egg <- c(1, 2, 3, 4)
```

**the code below is used to find  $k$  by making the sum of all probabilities equal to 1. The PMF is  $P(X = x) = k(2x + 1)$ .**

```
k <- 1 / sum(2 * egg + 1)
```

```
cat("Formula:") cat("Sum of $P(X = x) = 1$ ") cat(" $k(3 + 5 + 7 + 9) = 1$ ") cat(" $24k = 1$ ") cat(" $k = 1/24 =$ ", round(k, 4), "")
```

**below just calculates all the probability of all  $x$  possibilities**

```
P_X <- k * (2 * egg + 1)
```

```
cat("Probabilities of X :") cat(" $P(X = 1) = k(2(1) + 1) =$ ", round(P_X[1], 4), "") cat(" $P(X = 2) = k(2(2) + 1) =$ ", round(P_X[2], 4), "") cat(" $P(X = 3) = k(2(3) + 1) =$ ", round(P_X[3], 4), "") cat(" $P(X = 4) = k(2(4) + 1) =$ ", round(P_X[4], 4), "")
```

**Then we calculate the expected value  $E[X]$  which can be found by multiplying each value of  $X$  by its probability, and then, adding all the results together**

```
E_X <- sum(egg * P_X)
```

```
cat("Expected value formula:") cat("E[X] = Sum of xP(X = x)") cat("E[X] = 1(0.1250) + 2(0.2083) + 3(0.2917) + 4(0.3750)") cat("E[X] =", round(E_X, 4), "")
```

then we calculate for  $E[X^2]$  since is needed to calculate for variance formula.

```
E_X_squared <- sum((egg^2) * P_X)
```

```
cat("E[X^2] = Sum of x^2P(X = x)") cat("E[X^2] = 1^2(0.1250) + 2^2(0.2083) + 3^2(0.2917) + 4^2(0.3750)") cat("E[X^2] =", round(E_X_squared, 4), "")
```

this is the variance formula,  $\text{Var}(X) = E[X^2] - (E[X])^2$ .

```
Var_X <- E_X_squared - (E_X^2)
```

```
cat("Variance formula:") cat("Var(X) = E[X^2] - (E[X])^2") cat("Var(X) =", round(E_X_squared, 4), "- (", round(E_X, 4), ")^2") cat("Var(X) =", round(Var_X, 4), "")
```

lastly, to calculate  $P(X \geq 3)$  we add the probabilities of  $X = 3$  and  $X = 4$ .

```
P_X_greater_equal_3 <- sum(P_X[egg >= 3])
```

```
cat("Probability formula:") cat("P(X >= 3) = P(X = 3) + P(X = 4)") cat("P(X >= 3) = 0.2917 + 0.3750") cat("P(X >= 3) =", round(P_X_greater_equal_3, 4), "")
```

And so the final answers are below:

```
print("(A) Find the value of the constant k.") cat("The value is (A): k =", round(k, 4), "")
```

```
print("(B) Compute the expected value E[X] and the variance Var(X).") cat("The expected value and variance is (B): E[X] =", round(E_X, 4), "and Var(X) =", round(Var_X, 4), "")
```

```
print("(c) Calculate P(X ≥ 3).") cat("Calculation for (C): P(X >= 3) =", round(P_X_greater_equal_3, 4), "")
```

```
#T2. (Binomial Distribution) [7 Points] - An IT system detects that roughly 8% of incoming external requests are malicious. In a random sample of 20 requests
```

```
```{r exer2}
```

```
# P means probability, n is the number of requests, p is the probability that a request is malicious, and q is the probability that a request is not malicious.
```

```

n <- 20
p <- 0.08

q <- 1 - p

cat("Given:\n")
cat("n =", n, "\n")
cat("p =", p, "\n")
cat("q = 1 - p =", round(q, 4), "\n\n")

# this uses the binomial prob formula, and should calculate the
probability of exactly 2 malicious requests.
P_X_equal_2 <- choose(n, 2) * (p^2) * (q^(n - 2))

cat("Binomial probability formula:\n")
cat("P(X = x) = C(n,x) p^x (1-p)^(n-x)\n")
cat("P(X = 2) = C(20,2)(0.08)^2(0.92)^18\n")
cat("P(X = 2) =", round(P_X_equal_2, 4), "\n\n")

# This is going to be calculating the probability of at least 3
malicious requests. At least 3 means 3 or more. We use the complement:
P(X >= 3) = 1 - P(X <= 2).
P_X_greater_equal_3 <- 1 - pbinom(2, n, p)

# Then we need to calculate the probabilities thats needed for the
complement.
P_X_equal_0 <- choose(n, 0) * (p^0) * (q^n)
P_X_equal_1 <- choose(n, 1) * (p^1) * (q^(n - 1))
P_X_equal_2_again <- choose(n, 2) * (p^2) * (q^(n - 2))

cat("Complement formula:\n")
cat("P(X >= 3) = 1 - P(X <= 2)\n")
cat("P(X >= 3) = 1 - [P(X = 0) + P(X = 1) + P(X = 2)]\n")
cat("P(X >= 3) = 1 - [",
    round(P_X_equal_0, 4), " + ",
    round(P_X_equal_1, 4), " + ",
    round(P_X_equal_2_again, 4), "]\n")
cat("P(X >= 3) =", round(P_X_greater_equal_3, 4), "\n\n")

# This just calculate the mean of the binomial distribution, formula
being below
mean_X <- n * p

cat("Mean formula:\n")
cat("Mean = np\n")
cat("Mean = (20)(0.08)\n")
cat("Mean =", round(mean_X, 4), "\n\n")

# Then we just calculate the standard deviation. The formula is below

```

```

SD_X <- sqrt(n * p * q)

cat("Standard deviation formula:\n")
cat("SD = sqrt(np(1-p))\n")
cat("SD = sqrt((20)(0.08)(0.92))\n")
cat("SD =", round(SD_X, 4), "\n\n")

# And so the final answers are below:
print("(A) What is the probability that exactly 2 requests are
malicious?")
cat("The chances for exactly 2 requests being malicious is (A): P(X =
2) =", round(P_X_equal_2, 4), "\n")

print("(B) What is the probability that at least 3 requests are
malicious?")
cat("The chances for at least 3 requests being malicious is (B): P(X
>= 3) =", round(P_X_greater_equal_3, 4), "\n")

print("(C) Determine the mean and standard deviation of the number of
malicious requests.")
cat("The standard deviation and mean for the malicious requests are
(C): Mean =", round(mean_X, 4),
    "and Standard deviation =", round(SD_X, 4), "\n")

#Task number 3

```{r exer3}

```

**P is probability. Lambda (sandwich is just the R variable) is the average number of server hits per minute.**

```

sandwich <- 4.5

cat("Given:") cat("Lambda =", sandwich, "hits per minute")

```

**Using the Poisson probability formula we calculate the probability of exactly 6 hits in one minute.**

```

P_X_equal_6 <- (exp(-sandwich) * sandwich^6) / factorial(6)

cat("Poisson probability formula:") cat("P(X = x) = (e^(-lambda) * lambda^x) / x!") cat("P(X
= 6) = (e^(-4.5) * 4.5^6) / 6!") cat("P(X = 6) =", round(P_X_equal_6, 4), "")

```

and then this time we calculate the probability of at most 2 hits in one minute. At most 2 means 0, 1, 2 hits.

```
P_X_equal_0 <- (exp(-sandwich) * sandwich^0) / factorial(0) P_X_equal_1 <- (exp(-sandwich) * sandwich^1) / factorial(1) P_X_equal_2 <- (exp(-sandwich) * sandwich^2) / factorial(2)
```

```
P_X_at_most_2 <- P_X_equal_0 + P_X_equal_1 + P_X_equal_2
```

```
cat("Probability formula:") cat("P(X <= 2) = P(X = 0) + P(X = 1) + P(X = 2)") cat("P(X = 0) =", round(P_X_equal_0, 4), "") cat("P(X = 1) =", round(P_X_equal_1, 4), "") cat("P(X = 2) =", round(P_X_equal_2, 4), "") cat("P(X <= 2) =", round(P_X_equal_0, 4), "+", round(P_X_equal_1, 4), "+", round(P_X_equal_2, 4), "") cat("P(X <= 2) =", round(P_X_at_most_2, 4), "")
```

For a 2-minute interval, the average number of hits is doubled.

```
sandwich_2_minutes <- sandwich * 2
```

```
cat("For a 2-minute interval:") cat("Lambda = 4.5(2) =", sandwich_2_minutes, "")
```

To calculate the probability of more than 10 hits in 2 minutes, More than 10 means 11 or more, We use the complement below

```
P_X_at_most_10 <- sum((exp(-sandwich_2_minutes) * sandwich_2_minutes^(0:10)) / factorial(0:10))
```

```
P_X_greater_10 <- 1 - P_X_at_most_10
```

```
cat("Complement formula:") cat("P(X > 10) = 1 - P(X <= 10)") cat("P(X <= 10) = P(X = 0) + P(X = 1) + ... + P(X = 10)") cat("P(X <= 10) =", round(P_X_at_most_10, 4), "") cat("P(X > 10) = 1 -", round(P_X_at_most_10, 4), "") cat("P(X > 10) =", round(P_X_greater_10, 4), "")
```

And so the final answers are below:

```
print("(A) Find the probability of receiving exactly 6 hits in a given minute.") cat("The probability for receiving exactly 6 hits in a given minute is (A): P(X = 6) =", round(P_X_equal_6, 4), "")
```

```
print("(B) Find the probability of receiving at most 2 hits in a given minute.") cat("the probability of receiving at most 2 hits in a given minute is (B): P(X <= 2) =", round(P_X_at_most_2, 4), "")
```

```
print("(C) Calculate the probability of receiving more than 10 hits in a 2-minute interval.") cat("the probability of receiving more than 10 hits in a 2-minute interval is (C): P(X > 10) =", round(P_X_greater_10, 4), "")
```

```
#Number 4
```

```
Item Number 4
```

```
```{r exer4}
```

```
# The query time follows a normal distribution. mu is the mean and  
sigma is the standard deviation.
```

```
mu <- 120
```

```
sigma <- 15
```

```
cat("Given:\n")
```

```
cat("Mean (mu) =", mu, "ms\n")
```

```
cat("Standard deviation (sigma) =", sigma, "ms\n\n")
```

```
# We calculate the probability that the query time is longer than 135  
ms by calculating the z-score first
```

```
z_135 <- (135 - mu) / sigma
```

```
# then we have  $P(X > 135)$  which should be the area to the right of  
the z-score.
```

```
P_X_greater_135 <- 1 - pnorm(z_135)
```

```
cat("Part (A):\n")
```

```
cat("Z-score formula:\n")
```

```
cat("z = (X - mu) / sigma\n")
```

```
cat("z = (135 - 120) / 15\n")
```

```
cat("z =", round(z_135, 4), "\n")
```

```
cat("P(X > 135) = 1 - P(Z <= 1)\n")
```

```
cat("P(X > 135) =", round(P_X_greater_135, 4), "\n\n")
```

```
# Now we calculate the z-score for both values. and then subtract the  
area below the lower z-score from the area below the upper z-score.
```

```
z_100 <- (100 - mu) / sigma
```

```
z_130 <- (130 - mu) / sigma
```

```
P_X_between <- pnorm(z_130) - pnorm(z_100)
```

```
# and then calculate the probability that the query time is between  
100 ms and 130 ms.
```

```
cat("Part (B):\n")
```

```
cat("Lower z-score:\n")
```

```
cat("z = (100 - 120) / 15\n")
```

```
cat("z =", round(z_100, 4), "\n\n")
```

```
cat("Upper z-score:\n")
```

```
cat("z = (130 - 120) / 15\n")
```

```
cat("z =", round(z_130, 4), "\n\n")
```

```

cat("Probability formula:\n")
cat("P(100 < X < 130) = P(Z < 0.6667) - P(Z < -1.3333)\n")
cat("P(100 < X < 130) =", round(P_X_between, 4), "\n\n")

# qnorm(0.95) gives the z-score with 95% of the area below it.
z_95 <- qnorm(0.95)

# this just converts the z-score back to the original query time.
X_95 <- mu + z_95 * sigma

cat("Part (C):\n")
cat("95th percentile z-score = qnorm(0.95)\n")
cat("z =", round(z_95, 4), "\n")
cat("X = mu + z(sigma)\n")
cat("X = 120 + (", round(z_95, 4), ")(15)\n")
cat("X =", round(X_95, 4), "ms\n\n")

# And so the final answers are below:
print("(A) What proportion of queries take longer than 135
milliseconds to process?")
cat("The proportion of queries to take longer would be (A): P(X > 135)
=", round(P_X_greater_135, 4), "\n")

print("(B) Find the probability that a randomly chosen query takes
between 100 and 130 milliseconds.")
cat("The chances that a random chosen query takes place between 100
and 130 ms would be (B): P(100 < X < 130) =", round(P_X_between, 4),
"\n")

print("(C) Find the 95th percentile of query processing times.")
cat("The 95th percentile of query processing times would be (C): 95th
percentile =", round(X_95, 4), "ms\n")

```

Github link <https://github.com/Notcheesechess/APM1110-FA5>